

RENATHA

Pharmacy Inventory & Expiry Alert System

Full Project Developer Reference

Frontend (React + Vite) • Backend (Laravel) • Database Schema • API Reference

Stack: Laravel 11 | React + Vite | MySQL | Laravel Sanctum | Africa's Talking

Target: Small & Medium Pharmacies — East Africa

1. Project Overview

1.1 Abstract

RENATHA is a web-based Pharmacy Inventory & Expiry Alert System designed for small and medium pharmacies in East Africa. It solves three critical operational problems: sale of expired drugs due to absent tracking, poor stock visibility leading to shortages, and lack of affordable digital tooling in low-connectivity environments.

The system combines a Laravel REST API with a React.js single-page application, MySQL for storage, and Africa's Talking APIs for SMS/USSD — ensuring accessibility even without a stable internet connection.

1.2 Objectives

General Objective

Develop a digital pharmacy inventory system that improves drug safety and inventory efficiency in low-resource environments.

Specific Objectives

- Implement automated expiry tracking with configurable alert thresholds (30 / 14 / 7 days)
- Provide real-time inventory monitoring per drug and per batch
- Enable low-stock notifications when quantity falls below reorder level
- Integrate SMS and USSD for offline / low-connectivity access
- Develop a role-based user interface for Admin, Pharmacist, and Cashier
- Maintain accurate sales records with automatic stock deduction

1.3 Technology Stack

Layer	Technology	Purpose
Backend	Laravel (PHP 8.2+)	REST API, business logic, scheduled jobs
Frontend	React.js + Vite	SPA interface, dashboard, forms
Database	MySQL / PostgreSQL	Relational data storage
Auth	Laravel Sanctum	API token authentication
SMS / USSD	Africa's Talking	Alerts & offline access
Styling	Tailwind CSS	Utility-first responsive UI

1.4 Project Roadmap

Week	Phase	Tasks
1	Foundation	DB schema design, Laravel project, user auth, migrations
2	Core APIs	Drug, batch, sale endpoints; React scaffold, Axios setup
3	Alerts	Expiry scheduler, low-stock logic, SMS integration (Africa's Talking)
4	UI & USSD	Dashboard, drug/batch pages, basic USSD menu flow
5–6	Polish	Sales module, reports, role-based access, testing, deployment

2. Frontend Setup (React + Vite)

💡 *Target environment: Windows 10/11 — developer familiar with web dev but new to React.*

2.1 Prerequisites — Install Node.js

1. Go to nodejs.org and download the LTS version (v20.x or v22.x)
2. Run the installer, ensure 'Add to PATH' is checked
3. Open a new PowerShell window and verify:

```
node -v  
npm -v
```

Expected output: version numbers such as v20.x.x and 10.x.x

2.2 Install VS Code & Extensions

4. Download from code.visualstudio.com and install with defaults
5. Inside VS Code, install these extensions (Ctrl+Shift+X):
 1. ES7+ React/Redux Snippets
 2. Prettier – Code Formatter
 3. Tailwind CSS IntelliSense
 4. GitLens (optional)

2.3 Create the React Project

Open PowerShell, navigate to your projects folder, and run:

```
cd C:\Users\YourName\Projects  
npm create vite@latest renatha-frontend -- --template react  
cd renatha-frontend  
npm install
```

2.4 Install Core Libraries

```
npm install react-router-dom  
npm install axios  
npm install -D tailwindcss postcss autoprefixer  
npx tailwindcss init -p  
npm install lucide-react  
npm install recharts  
npm install date-fns
```

2.5 Configure Tailwind CSS

Open `tailwind.config.js` and replace all content with:

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: { extend: {} },
  plugins: [],
}
```

Then open `src/index.css` and replace everything with:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

2.6 Folder Structure

Create the following directories inside `src/`:

```
src/
├── api/           ← Axios config & API call functions
├── components/    ← Reusable UI (Button, Table, Modal, Badge)
├── pages/         ← Full pages (Dashboard, Drugs, Batches, Sales, Alerts)
├── layouts/       ← Sidebar + main wrapper
├── context/       ← Auth state (login/logout)
└── utils/         ← Date formatting, helpers
```

Create via PowerShell (from inside the `src` folder):

```
mkdir api, components, pages, layouts, context, utils
```

2.7 Configure Axios (`src/api/axios.js`)

Create the file `src/api/axios.js`:

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8000/api',
  headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
  withCredentials: true,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

2.8 Set Up Routing (src/App.jsx)

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Dashboard from './pages/Dashboard';
import Drugs from './pages/Drugs';
import Sales from './pages/Sales';
import Alerts from './pages/Alerts';

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/drugs" element={<Drugs />} />
        <Route path="/sales" element={<Sales />} />
        <Route path="/alerts" element={<Alerts />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Create a placeholder for each page, for example src/pages/Dashboard.jsx:

```
export default function Dashboard() {
  return <div className='p-6 text-2xl font-bold'>Dashboard</div>;
}
```

2.9 Run the Dev Server

```
npm run dev
```

Open <http://localhost:5173> in your browser. You should see your app running.

2.10 Frontend Setup Checklist

Step	Action	Done?
1	Install Node.js LTS from nodejs.org	☐
2	Verify: node -v && npm -v in PowerShell	☐
3	Install VS Code + React/Prettier/Tailwind extensions	☐
4	Run: npm create vite@latest renatha-frontend -- --template react	☐
5	Install: react-router-dom, axios, tailwindcss, lucide-react, recharts, date-fns	☐

6	Configure tailwind.config.js and src/index.css	☐
7	Create folder structure: api/ components/ pages/ layouts/ context/ utils/	☐
8	Create src/api/axios.js with base URL + token interceptor	☐
9	Set up App.jsx with react-router-dom routes	☐
10	Create placeholder pages: Dashboard, Drugs, Sales, Alerts	☐
11	Run: npm run dev — confirm localhost:5173 loads	☐

3. Backend Setup (Laravel)

 Requires XAMPP for PHP and MySQL on Windows.

3.1 Install PHP via XAMPP

6. Download XAMPP from apacheFriends.org (PHP 8.2+)
7. Install to default path C:\xampp
8. Add PHP to system PATH:
 5. Search 'Environment Variables' in Windows Start
 6. System Variables → Path → Edit → New → add C:\xampp\php
 7. Click OK on all windows
9. Verify in a NEW PowerShell window:

```
php -v
```

3.2 Install Composer

10. Download Composer-Setup.exe from getcomposer.org/download
11. Run installer — it will auto-detect C:\xampp\php\php.exe
12. Complete with defaults, then verify:

```
composer -v
```

3.3 Set Up MySQL Database

13. Open XAMPP Control Panel and Start both Apache and MySQL
14. Open <http://localhost/phpmyadmin> in your browser
15. Click New in the left sidebar
16. Enter database name: renatha_db
17. Set collation: utf8mb4_unicode_ci
18. Click Create

3.4 Create Laravel Project

```
cd C:\Users\YourName\Projects
composer create-project laravel/laravel renatha-backend
cd renatha-backend
php artisan serve
```

Visit <http://localhost:8000> — you should see the Laravel welcome page.

3.5 Configure .env

Open renatha-backend/.env in VS Code and set:


```
APP_NAME=RENATHA
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=renatha_db
DB_USERNAME=root
DB_PASSWORD=
```

```
php artisan key:generate
```

3.6 Install Laravel Sanctum

```
composer require laravel/sanctum
php artisan install:api
php artisan migrate
```

Add HasApiTokens to app/Models/User.php:

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable {
    use HasApiTokens, HasFactory, Notifiable;

    protected $fillable = ['name', 'email', 'password', 'role', 'phone'];
}
```

3.7 Run Migrations

Generate migration files:

```
php artisan make:migration create_drugs_table
php artisan make:migration create_batches_table
php artisan make:migration create_sales_table
php artisan make:migration create_alerts_table
php artisan make:migration add_role_phone_to_users_table
```

Fill each migration file (see Section 4 — Database Schema for full column details), then run:

```
php artisan migrate
```

3.8 Create Models

```
php artisan make:model Drug
php artisan make:model Batch
php artisan make:model Sale
php artisan make:model Alert
```

Each model needs a \$fillable array and relationships. Example for Batch:

```
class Batch extends Model {
    protected $fillable = [
        'drug_id', 'batch_number', 'quantity',
        'buying_price', 'selling_price', 'expiry_date', 'manufacture_date'
    ];
    public function drug() { return $this->belongsTo(Drug::class); }
    public function sales() { return $this->hasMany(Sale::class); }
}
```

3.9 Create API Controllers

```
php artisan make:controller Api/AuthController
php artisan make:controller Api/DrugController --resource
php artisan make:controller Api/BatchController --resource
php artisan make:controller Api/SaleController --resource
php artisan make:controller Api/AlertController
```

3.10 Define API Routes (routes/api.php)

```
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::apiResource('drugs', DrugController::class);
    Route::apiResource('batches', BatchController::class);
    Route::apiResource('sales', SaleController::class);
    Route::get('/alerts', [AlertController::class, 'index']);
    Route::patch('/alerts/{id}/read', [AlertController::class, 'markRead']);
});
```

3.11 Configure CORS

Open config/cors.php and update:

```
'allowed_origins' => ['http://localhost:5173'],
'allowed_methods' => ['*'],
'allowed_headers' => ['*'],
'supports_credentials' => true,
```

3.12 Backend Setup Checklist

Step	Action	Done?
------	--------	-------

1	Install XAMPP (PHP 8.2+) and add C:\xampp\php to system PATH	☐
2	Verify: php -v in a new PowerShell window	☐
3	Install Composer from getcomposer.org	☐
4	Start XAMPP Apache + MySQL, create renatha_db in phpMyAdmin	☐
5	Run: composer create-project laravel/laravel renatha-backend	☐
6	Configure .env: DB_DATABASE=renatha_db, DB_USERNAME=root, DB_PASSWORD=	☐
7	Run: composer require laravel/sanctum && php artisan install:api	☐
8	Create all migrations (drugs, batches, sales, alerts, add role/phone to users)	☐
9	Run: php artisan migrate	☐
10	Create Models with fillable fields and relationships	☐
11	Create API Controllers: Auth, Drug, Batch, Sale, Alert	☐
12	Define routes in routes/api.php (public + sanctum-protected)	☐
13	Configure config/cors.php for localhost:5173	☐
14	Test login endpoint in Postman — confirm token returned	☐

4. Database Schema

All tables use Laravel's auto-managed `created_at` and `updated_at` timestamps. Foreign keys use `onDelete('cascade')` unless noted.

4.1 users

Column	Type	Notes
<code>id</code>	BIGINT PK	Auto-increment primary key
<code>name</code>	VARCHAR(255)	Full name
<code>email</code>	VARCHAR(255)	Unique login email
<code>password</code>	VARCHAR(255)	Bcrypt hashed
<code>role</code>	ENUM	admin pharmacist cashier
<code>phone</code>	VARCHAR(20)	For SMS alerts via Africa's Talking
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	Laravel auto-managed

4.2 drugs

Column	Type	Notes
<code>id</code>	BIGINT PK	Auto-increment primary key
<code>name</code>	VARCHAR(255)	Drug name e.g. Paracetamol 500mg
<code>category</code>	VARCHAR(100)	e.g. Antibiotic, Analgesic
<code>unit</code>	VARCHAR(50)	tablet, bottle, vial, sachet
<code>reorder_level</code>	INT	Min quantity before low-stock alert (default 10)
<code>is_active</code>	BOOLEAN	Soft delete flag (default true)
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	Laravel auto-managed

4.3 batches

Column	Type	Notes
--------	------	-------

id	BIGINT PK	Auto-increment primary key
drug_id	FK → drugs	Links batch to drug
batch_number	VARCHAR(100)	Manufacturer batch number
quantity	INT	Current stock in this batch
buying_price	DECIMAL(10,2)	Cost per unit
selling_price	DECIMAL(10,2)	Retail price per unit
expiry_date	DATE	Critical — triggers expiry alerts
manufacture_date	DATE	Optional
created_at / updated_at	TIMESTAMP	Laravel auto-managed

4.4 sales

Column	Type	Notes
id	BIGINT PK	Auto-increment primary key
batch_id	FK → batches	Which batch was sold from
user_id	FK → users	Who made the sale
quantity_sold	INT	Units sold; deducted from batch.quantity
total_price	DECIMAL(10,2)	quantity_sold × selling_price
created_at / updated_at	TIMESTAMP	Laravel auto-managed

4.5 alerts

Column	Type	Notes
id	BIGINT PK	Auto-increment primary key
drug_id	FK → drugs	Which drug triggered the alert
type	ENUM	expiry low_stock
message	TEXT	Human-readable alert description
status	ENUM	unread read (default unread)
created_at / updated_at	TIMESTAMP	Laravel auto-managed

4.6 Entity Relationships

- One Drug has many Batches
- One Batch has many Sales
- One User has many Sales
- One Drug has many Alerts
- A Sale deducts quantity from its Batch at creation

5. API Reference

Base URL: `http://localhost:8000/api`

All protected endpoints require the header: `Authorization: Bearer {token}`

💡 Test all endpoints in Postman. For protected routes use the token returned from `/api/login`.

5.1 Authentication

Method	Endpoint	Auth	Description
POST	<code>/api/register</code>	Public	Register a new user
POST	<code>/api/login</code>	Public	Login, returns bearer token
POST	<code>/api/logout</code>	Token	Invalidate current token

5.2 Drugs

Method	Endpoint	Auth	Description
GET	<code>/api/drugs</code>	Token	List all drugs
POST	<code>/api/drugs</code>	Token	Create a new drug
GET	<code>/api/drugs/{id}</code>	Token	Get single drug details
PUT	<code>/api/drugs/{id}</code>	Token	Update drug record
DELETE	<code>/api/drugs/{id}</code>	Token	Soft-delete a drug

5.3 Batches

Method	Endpoint	Auth	Description
GET	<code>/api/batches</code>	Token	List all batches (with drug info)
POST	<code>/api/batches</code>	Token	Add new stock batch
GET	<code>/api/batches/{id}</code>	Token	Get batch details
PUT	<code>/api/batches/{id}</code>	Token	Update batch (qty, price, etc.)
DELETE	<code>/api/batches/{id}</code>	Token	Delete batch

5.4 Sales

Method	Endpoint	Auth	Description
GET	/api/sales	Token	List all sales records
POST	/api/sales	Token	Record a new sale (deducts stock)
GET	/api/sales/{id}	Token	Get single sale details

5.5 Alerts

Method	Endpoint	Auth	Description
GET	/api/alerts	Token	List all alerts (filter by status)
PATCH	/api/alerts/{id}/read	Token	Mark alert as read

5.6 Example: Login Request & Response

POST /api/login — Request body:

```
{ "email": "admin@renatha.com", "password": "secret123" }
```

Successful response (200):

```
{ "user": { "id": 1, "name": "Admin", "role": "admin" },  
  "token": "1|abc123xyz..." }
```

Failed response (401):

```
{ "message": "Invalid credentials" }
```


6. Africa's Talking Integration

6.1 Setup

19. Register at africastalking.com — a free sandbox is available
20. From your dashboard, collect: API Key, Username, Sender ID / Shortcode
21. Install the PHP SDK in your Laravel project:

```
composer require africastalking/africastalking
```

22. Add credentials to .env:

```
AT_USERNAME=sandbox
AT_API_KEY=your_api_key_here
```

6.2 SMS Alert Logic

Create a Laravel scheduled command (app/Console/Commands/CheckExpiryAlerts.php) that runs daily:

- Queries batches where expiry_date is 30, 14, or 7 days away
- Queries batches where quantity <= drug.reorder_level
- Sends an SMS to all users with role admin or pharmacist
- Creates an entry in the alerts table

Register the command in app/Console/Kernel.php:

```
protected function schedule(Schedule $schedule) {
    $schedule->command('alerts:check-expiry')->daily();
}
```

6.3 USSD Menu Flow

Design your USSD menu tree for the shortcode (e.g. *384#):

```
CON Welcome to RENATHA
1. Check Stock
2. View Active Alerts
3. Record Quick Sale
```

- Response prefix CON = continues the session
- Response prefix END = terminates the session
- Handle via a POST route in Laravel: `Route::post('/ussd', [UssdController::class, 'handle'])`

7. Next Steps After Setup

Once both frontend and backend are running and the login API returns a token, work through these milestones in order:

23. Connect React login form to POST /api/login and store token in localStorage
24. Build Drug management page: list, add, edit drugs
25. Build Batch management: add stock with expiry date
26. Build Sales page: select drug, enter quantity, auto-calculate price
27. Build Alerts dashboard: show unread alerts, mark as read
28. Build Dashboard: total drugs, low stock count, expiring soon count, charts
29. Implement role-based route guards in React (admin vs cashier views)
30. Implement Africa's Talking SMS scheduler
31. Design and implement USSD controller
32. Deploy to a VPS (DigitalOcean or Hetzner) with Nginx + MySQL + SSL

 *Keep this document updated as the project evolves. Add new API endpoints and schema changes here as they are built.*